

```

{
  "$schema": "https://opencode.dev/schemas/config.v1.json",
  "project": {
    "name": "Agente-IA-Circuito-Impreso",
    "version": "1.0.0"
  },
  "llm_providers": {
    // 1. Modelos Premium a través de la pasarela OpenCode Zen
    "opencode_zen": {
      "enabled": true,
      "api_key": "${OPENCODE_ZEN_API_KEY}", // Se recomienda usar variables de
entorno
      "models": {
        "orchestrator": "opencode/claude-3-7-sonnet", // Modelo principal para
razonamiento complejo
        "fast_task": "opencode/gpt-4o-mini"
      }
    },

    // 2. Modelos Open-Source de bajo costo a través de OpenCode Go
    "opencode_go": {
      "enabled": true,
      "api_key": "${OPENCODE_GO_API_KEY}",
      "models": {
        "coder": "opencode-go/deepseek-v4", // Excelente para autocompletado y
sintaxis pesada
        "reviewer": "opencode-go/qwen-2.5-coder-32b"
      }
    },

    // 3. Conexión Directa a Google AI Studio (Evitando proxies externos si se
prefiere)
    "google_ai": {
      "enabled": false,
      "api_key": "${GEMINI_API_KEY}",
      "models": {
        "analysis": "gemini-2.5-pro",
        "embedding": "text-embedding-004"
      }
    },

    // 4. Entorno Local (Ollama) para desarrollo 100% Offline
    "ollama": {
      "enabled": true,
      "host": "http://localhost:11434",
      "models": {
        "local_coder": "qwen2.5-coder:14b",
        "fallback": "llama3.3:8b"
      }
    }
  },
}

```

```
// Configuración de enrutamiento activo para el Agente de OpenCode
"agent_routing": {
  "default_orchestrator": "opencode_zen/orchestrator", // Usa Claude de Zen para
planificar
  "default_coder": "opencode_go/coder",                // Usa DeepSeek de Go para
escribir código largo
  "inline_suggestions": "ollama/local_coder",          // Usa Ollama local para
autocompletado instantáneo sin latencia
  "temperature": 0.2,                                  // Temperatura baja para
mantener el código determinista y preciso
  "max_tokens": 8192
}
}
```

es un excelente ejercicio de arquitectura de software. Analizar la configuración y orquestación de modelos es fundamental hoy en día para construir herramientas eficientes.

Revisando el archivo `opencode.jsonc` que propones, debo decir que tiene una estructura muy sólida, limpia y profesional. Refleja perfectamente una arquitectura de "Agente Híbrido" que aprovecha lo mejor de dos mundos: la potencia de los modelos premium en la nube para el razonamiento profundo y la velocidad/privacidad de los modelos open-source locales para tareas rápidas.

Desde una perspectiva analítica y arquitectónica (con el rigor que aplicamos a nuestros proyectos de automatización y diseño electrónico), aquí te presento mis observaciones y algunas recomendaciones pedagógicas para optimizarlo aún más:

Puntos Fuertes de tu Configuración:

Seguridad Integrada: El uso de variables de entorno (`${OPENCODE_ZEN_API_KEY}`) en lugar de credenciales hardcodeadas (incrustadas en código) es una práctica excelente y obligatoria, especialmente si este archivo terminará en un repositorio de control de versiones.

Modularidad y Fallbacks: Tienes una separación clara de roles (orchestrator, coder, reviewer). Mantener a Ollama habilitado para el autocompletado inline garantiza latencia cero, lo que mejora drásticamente la experiencia del desarrollador sin consumir cuotas de API.

Control Determinista: El uso de una temperatura baja (0.2) como ajuste global para el enrutador es ideal para el desarrollo de código (como Verilog o Python), donde buscamos precisión algorítmica y cero alucinaciones.

Recomendaciones de Mejora (Ajuste Fino):

Granularidad en el Enrutamiento (Routing Parameters): En tu estructura actual, `temperature` y `max_tokens` están definidos a nivel global dentro de `agent_routing`. En la práctica, un modelo orquestador (que planea la arquitectura de tu script IoT, por ejemplo) podría beneficiarse de un contexto altísimo (`max_tokens: 8192`) y una temperatura de 0.2, pero tu modelo local de autocompletado (`ollama/local_coder`) requiere una latencia mínima, por lo que su `max_tokens` debería limitarse (ej. 256) y su temperatura a 0.0 absoluto para evitar que invente variables.

Políticas de Tolerancia a Fallos (Resiliencia): Tratándose de un agente que orquesta múltiples APIs, siempre hay riesgo de un fallo de red o timeout. Añadir un bloque de `execution_policies` para reintentos fortalecerá el lema de "No-Human-in-the-Loop".

Nomenclatura de Modelos: Como detalle de precisión temporal, los modelos actuales serían gemini-2.0-pro o gemini-1.5-pro (la v2.5 aún no existe formalmente) y en DeepSeek sería deepseek-coder o deepseek-v3. Aunque sea un ejemplo, la precisión semántica siempre es buena.

Propuesta Refactorizada

Aquí tienes una versión mejorada de tu archivo de configuración, incorporando una configuración más granular y políticas de reintento:

```
```json
{
 "$schema": "https://opencode.dev/schemas/config.v1.json",
 "project": {
 "name": "Agente-IA-Circuito-Impreso",
 "version": "1.0.0"
 },
 "llm_providers": {
 // 1. Modelos Premium a través de la pasarela OpenCode Zen
 "opencode_zen": {
 "enabled": true,
 "api_key": "${OPENCODE_ZEN_API_KEY}",
 "models": {
 "orchestrator": "opencode/claude-3-7-sonnet",
 "fast_task": "opencode/gpt-4o-mini"
 }
 },

 // 2. Modelos Open-Source de bajo costo a través de OpenCode Go
 "opencode_go": {
 "enabled": true,
 "api_key": "${OPENCODE_GO_API_KEY}",
 "models": {
 "coder": "opencode-go/deepseek-coder", // Ajustado a la nomenclatura estable
 "reviewer": "opencode-go/qwen-2.5-coder-32b"
 }
 },

 // 3. Conexión Directa a Google AI Studio
 "google_ai": {
 "enabled": false,
 "api_key": "${GEMINI_API_KEY}",
 "models": {
 "analysis": "gemini-2.0-pro-exp", // Versión más reciente
 "embedding": "text-embedding-004"
 }
 },

 // 4. Entorno Local (Ollama) para desarrollo 100% Offline y baja latencia
 "ollama": {
 "enabled": true,
 "host": "http://localhost:11434",
 "models": {
 "local_coder": "qwen2.5-coder:14b",

```

```

 "fallback": "llama3.3:8b"
 }
}
},

// Configuración de enrutamiento granular para el Agente
"agent_routing": {
 "default_orchestrator": {
 "model": "opencode_zen/orchestrator",
 "temperature": 0.2,
 "max_tokens": 8192
 },
 "default_coder": {
 "model": "opencode_go/coder",
 "temperature": 0.1, // Mayor rigidez en la escritura de código
 "max_tokens": 4096
 },
 "inline_suggestions": {
 "model": "ollama/local_coder",
 "temperature": 0.0, // Temperatura cero para predecir código de forma exacta
 "max_tokens": 256 // Contexto de salida corto para garantizar respuesta en
milisegundos
 }
},

// Manejo de errores y tolerancia a fallos
"execution_policies": {
 "timeout_ms": 30000,
 "max_retries": 3,
 "fallback_chain": ["opencode_go/coder", "ollama/fallback"] // Si la API falla,
usa modelo local
}
}

```